In [ ]:
```
Objective
```

Delhivery is the largest and fastest-growing fully integrated player in India by revenue in Fiscal 2021. They aim to build the operating system for commerce, through a combination of world-class infrastructure, logistics operations of the highest quality, and cutting-edge engineering and technology capabilities.

The Data team builds intelligence and capabilities using this data that helps them to widen the gap between the quality, efficiency, and profitability of their business versus their competitors.

The company wants to understand and process the data coming out of data engineering pipelines:

• Clean, sanitize and manipulate data to get useful features out of raw fields

• Make sense out of the raw data and help the data science team to build forecasting models on it

In [247…
```
import pandas as pd
```

In [248…
```
df=pd.read_csv('delhivery_data.csv')
```

Analyzing structure & characteristics of the dataset.

In [249…
```
df.shape
```

Out[249…
```
(144867, 24)
```

In [250…
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 144867 entries, 0 to 144866
Data columns (total 24 columns):
 #   Column                          Non-Null Count   Dtype
---  ------                          --------------   -----
 0   data                            144867 non-null  object
 1   trip_creation_time              144867 non-null  object
 2   route_schedule_uuid             144867 non-null  object
 3   route_type                      144867 non-null  object
 4   trip_uuid                       144867 non-null  object
 5   source_center                   144867 non-null  object
 6   source_name                     144574 non-null  object
 7   destination_center              144867 non-null  object
 8   destination_name                144606 non-null  object
 9   od_start_time                   144867 non-null  object
 10  od_end_time                     144867 non-null  object
 11  start_scan_to_end_scan          144867 non-null  float64
 12  is_cutoff                       144867 non-null  bool
 13  cutoff_factor                   144867 non-null  int64
 14  cutoff_timestamp                144867 non-null  object
 15  actual_distance_to_destination  144867 non-null  float64
 16  actual_time                     144867 non-null  float64
 17  osrm_time                       144867 non-null  float64
 18  osrm_distance                   144867 non-null  float64
 19  factor                          144867 non-null  float64
 20  segment_actual_time             144867 non-null  float64
 21  segment_osrm_time               144867 non-null  float64
 22  segment_osrm_distance           144867 non-null  float64
 23  segment_factor                  144867 non-null  float64
dtypes: bool(1), float64(10), int64(1), object(12)
memory usage: 25.6+ MB
```

In [251…

```
df.describe()
```

Out[251…

	start_scan_to_end_scan	cutoff_factor	actual_distance_to_destination	
count	144867.000000	144867.000000	144867.000000	144
mean	961.262986	232.926567	234.073372	
std	1037.012769	344.755577	344.990009	
min	20.000000	9.000000	9.000045	
25%	161.000000	22.000000	23.355874	
50%	449.000000	66.000000	66.126571	
75%	1634.000000	286.000000	286.708875	
max	7898.000000	1927.000000	1927.447705	4

In [252…

```
df.dtypes
```

Out[252…

	0
data	object
trip_creation_time	object
route_schedule_uuid	object
route_type	object
trip_uuid	object
source_center	object
source_name	object
destination_center	object
destination_name	object
od_start_time	object
od_end_time	object
start_scan_to_end_scan	float64
is_cutoff	bool
cutoff_factor	int64
cutoff_timestamp	object
actual_distance_to_destination	float64
actual_time	float64
osrm_time	float64
osrm_distance	float64
factor	float64
segment_actual_time	float64
segment_osrm_time	float64
segment_osrm_distance	float64
segment_factor	float64

dtype: object

In [253…

```
df
```

Out [253…

	data	trip_creation_time	route_schedule_uuid	route_type	
0	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	15374109
1	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	15374109
2	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	15374109
3	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	15374109
4	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	15374109
...	...	...	...	...	
144862	training	2018-09-20 16:24:28.436231	thanos::sroute:f0569d2f-4e20-4c31-8542-67b86d5...	Carting	15374606
144863	training	2018-09-20 16:24:28.436231	thanos::sroute:f0569d2f-4e20-4c31-8542-67b86d5...	Carting	15374606
144864	training	2018-09-20 16:24:28.436231	thanos::sroute:f0569d2f-4e20-4c31-8542-67b86d5...	Carting	15374606
144865	training	2018-09-20 16:24:28.436231	thanos::sroute:f0569d2f-4e20-4c31-8542-67b86d5...	Carting	15374606
144866	training	2018-09-20 16:24:28.436231	thanos::sroute:f0569d2f-4e20-4c31-8542-67b86d5...	Carting	15374606

144867 rows × 24 columns

Converting time columns into pandas datetime.

In [254…
```
df['trip_creation_time']=pd.to_datetime(df['trip_creation_time'],)
```

In [255…
```
df['od_start_time']=pd.to_datetime(df['od_start_time'])
df['od_end_time']=pd.to_datetime(df['od_end_time'])
```

In [256…
```
df['cutoff_timestamp']=pd.to_datetime(df['cutoff_timestamp'],format='mixe
```

In [257…
```
df.dtypes
```

Out[257…

	0
data	object
trip_creation_time	datetime64[ns]
route_schedule_uuid	object
route_type	object
trip_uuid	object
source_center	object
source_name	object
destination_center	object
destination_name	object
od_start_time	datetime64[ns]
od_end_time	datetime64[ns]
start_scan_to_end_scan	float64
is_cutoff	bool
cutoff_factor	int64
cutoff_timestamp	datetime64[ns]
actual_distance_to_destination	float64
actual_time	float64
osrm_time	float64
osrm_distance	float64
factor	float64
segment_actual_time	float64
segment_osrm_time	float64
segment_osrm_distance	float64
segment_factor	float64

dtype: object

In [258…

```
df
```

Out[258…

	data	trip_creation_time	route_schedule_uuid	route_type	
0	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	15374109
1	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	15374109
2	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	15374109
3	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	15374109
4	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	15374109
...	...	...	...	...	
144862	training	2018-09-20 16:24:28.436231	thanos::sroute:f0569d2f-4e20-4c31-8542-67b86d5...	Carting	15374606
144863	training	2018-09-20 16:24:28.436231	thanos::sroute:f0569d2f-4e20-4c31-8542-67b86d5...	Carting	15374606
144864	training	2018-09-20 16:24:28.436231	thanos::sroute:f0569d2f-4e20-4c31-8542-67b86d5...	Carting	15374606
144865	training	2018-09-20 16:24:28.436231	thanos::sroute:f0569d2f-4e20-4c31-8542-67b86d5...	Carting	15374606
144866	training	2018-09-20 16:24:28.436231	thanos::sroute:f0569d2f-4e20-4c31-8542-67b86d5...	Carting	15374606

144867 rows × 24 columns

Handling missing values

In [259…

```
df.isna().sum()
```

Out[259…

	0
data	0
trip_creation_time	0
route_schedule_uuid	0
route_type	0
trip_uuid	0
source_center	0
source_name	293
destination_center	0
destination_name	261
od_start_time	0
od_end_time	0
start_scan_to_end_scan	0
is_cutoff	0
cutoff_factor	0
cutoff_timestamp	0
actual_distance_to_destination	0
actual_time	0
osrm_time	0
osrm_distance	0
factor	0
segment_actual_time	0
segment_osrm_time	0
segment_osrm_distance	0
segment_factor	0

dtype: int64

In [260…

```
df[df.duplicated()]
```

Out[260…

data	trip_creation_time	route_schedule_uuid	route_type	trip_uuid	source_cent

0 rows × 24 columns

In [261…

```
df.dropna()
```

Out[261…

	data	trip_creation_time	route_schedule_uuid	route_type	
0	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	15374109
1	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	15374109
2	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	15374109
3	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	15374109
4	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	15374109
...	...	...	...	...	
144862	training	2018-09-20 16:24:28.436231	thanos::sroute:f0569d2f-4e20-4c31-8542-67b86d5...	Carting	15374606
144863	training	2018-09-20 16:24:28.436231	thanos::sroute:f0569d2f-4e20-4c31-8542-67b86d5...	Carting	15374606
144864	training	2018-09-20 16:24:28.436231	thanos::sroute:f0569d2f-4e20-4c31-8542-67b86d5...	Carting	15374606
144865	training	2018-09-20 16:24:28.436231	thanos::sroute:f0569d2f-4e20-4c31-8542-67b86d5...	Carting	15374606
144866	training	2018-09-20 16:24:28.436231	thanos::sroute:f0569d2f-4e20-4c31-8542-67b86d5...	Carting	15374606

144316 rows × 24 columns

In [262…

```
df
```

Out [262…

	data	trip_creation_time	route_schedule_uuid	route_type	
0	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	15374109
1	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	15374109
2	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	15374109
3	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	15374109
4	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	15374109
...	...	...	...	...	
144862	training	2018-09-20 16:24:28.436231	thanos::sroute:f0569d2f-4e20-4c31-8542-67b86d5...	Carting	15374606
144863	training	2018-09-20 16:24:28.436231	thanos::sroute:f0569d2f-4e20-4c31-8542-67b86d5...	Carting	15374606
144864	training	2018-09-20 16:24:28.436231	thanos::sroute:f0569d2f-4e20-4c31-8542-67b86d5...	Carting	15374606
144865	training	2018-09-20 16:24:28.436231	thanos::sroute:f0569d2f-4e20-4c31-8542-67b86d5...	Carting	15374606
144866	training	2018-09-20 16:24:28.436231	thanos::sroute:f0569d2f-4e20-4c31-8542-67b86d5...	Carting	15374606

144867 rows × 24 columns

In [263…

```
df['source_name']=df['source_name'].fillna('unknown')
df['destination_name']=df['destination_name'].fillna('inknown')
```

In [264…

```
df.isna().sum()
```

Out[264…

	0
data	0
trip_creation_time	0
route_schedule_uuid	0
route_type	0
trip_uuid	0
source_center	0
source_name	0
destination_center	0
destination_name	0
od_start_time	0
od_end_time	0
start_scan_to_end_scan	0
is_cutoff	0
cutoff_factor	0
cutoff_timestamp	0
actual_distance_to_destination	0
actual_time	0
osrm_time	0
osrm_distance	0
factor	0
segment_actual_time	0
segment_osrm_time	0
segment_osrm_distance	0
segment_factor	0

dtype: int64

Dropping duplicate rows if any

In [265…

```
df.dropna(how='all')
```

Out[265…

	data	trip_creation_time	route_schedule_uuid	route_type	
0	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	15374109
1	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	15374109
2	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	15374109
3	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	15374109
4	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	15374109
...	...	...	...	...	
144862	training	2018-09-20 16:24:28.436231	thanos::sroute:f0569d2f-4e20-4c31-8542-67b86d5...	Carting	15374606
144863	training	2018-09-20 16:24:28.436231	thanos::sroute:f0569d2f-4e20-4c31-8542-67b86d5...	Carting	15374606
144864	training	2018-09-20 16:24:28.436231	thanos::sroute:f0569d2f-4e20-4c31-8542-67b86d5...	Carting	15374606
144865	training	2018-09-20 16:24:28.436231	thanos::sroute:f0569d2f-4e20-4c31-8542-67b86d5...	Carting	15374606
144866	training	2018-09-20 16:24:28.436231	thanos::sroute:f0569d2f-4e20-4c31-8542-67b86d5...	Carting	15374606

144867 rows × 24 columns

In [266…

```
df['segment_key']=(df['trip_uuid'].astype(str)+'_'+df['source_center'].as
```

In [267…

```
df
```

Out [267…

	data	trip_creation_time	route_schedule_uuid	route_type	
0	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	15374109
1	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	15374109
2	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	15374109
3	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	15374109
4	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	15374109
...	...	...	...	...	
144862	training	2018-09-20 16:24:28.436231	thanos::sroute:f0569d2f-4e20-4c31-8542-67b86d5...	Carting	15374606
144863	training	2018-09-20 16:24:28.436231	thanos::sroute:f0569d2f-4e20-4c31-8542-67b86d5...	Carting	15374606
144864	training	2018-09-20 16:24:28.436231	thanos::sroute:f0569d2f-4e20-4c31-8542-67b86d5...	Carting	15374606
144865	training	2018-09-20 16:24:28.436231	thanos::sroute:f0569d2f-4e20-4c31-8542-67b86d5...	Carting	15374606
144866	training	2018-09-20 16:24:28.436231	thanos::sroute:f0569d2f-4e20-4c31-8542-67b86d5...	Carting	15374606

144867 rows × 25 columns

In [268…

```
df['segment_actual_time_sum']=df.groupby('segment_key')['segment_actual_t
df['segment_osrm_distance_sum'] = df.groupby('segment_key')['segment_osrm
df['segment_osrm_time_sum'] = df.groupby('segment_key')['segment_osrm_tim
```

In [269…

```
df
```

Out[269…

	data	trip_creation_time	route_schedule_uuid	route_type	
0	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	15374109
1	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	15374109
2	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	15374109
3	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	15374109
4	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	15374109
...	...	...	...	...	
144862	training	2018-09-20 16:24:28.436231	thanos::sroute:f0569d2f-4e20-4c31-8542-67b86d5...	Carting	15374606
144863	training	2018-09-20 16:24:28.436231	thanos::sroute:f0569d2f-4e20-4c31-8542-67b86d5...	Carting	15374606
144864	training	2018-09-20 16:24:28.436231	thanos::sroute:f0569d2f-4e20-4c31-8542-67b86d5...	Carting	15374606
144865	training	2018-09-20 16:24:28.436231	thanos::sroute:f0569d2f-4e20-4c31-8542-67b86d5...	Carting	15374606
144866	training	2018-09-20 16:24:28.436231	thanos::sroute:f0569d2f-4e20-4c31-8542-67b86d5...	Carting	15374606

144867 rows × 28 columns

In [270…

```
# create_segment_dict = {
#     "trip_uuid": "first",
#     "source_center": "first",
#     "destination_center": "first",
#     "segment_actual_time": "sum",
#     "segment_osrm_distance": "sum",
#     "segment_osrm_time": "sum",
#     "od_start_time": "first",
#     "od_end_time": "last",


# }
```

In [271…

```
# dictionary for aggregation at segment level
create_segment_dict = {
    'trip_creation_time': 'first',
```

```
    'route_schedule_uuid': 'first',
    'route_type': 'first',
    'trip_uuid': 'first',
    'source_center': 'first',
    'source_name': 'first',
    'destination_center': 'first',
    'destination_name': 'first',
    'od_start_time': 'min',     # segment starts
    'od_end_time': 'max',       # segment ends
    'start_scan_to_end_scan': 'first',
    'is_cutoff': 'max',
    'cutoff_factor': 'first',
    'cutoff_timestamp': 'first',
    'actual_distance_to_destination': 'max',

    # Trip/segment time & distance metrics
    'actual_time': 'sum',
    'osrm_time': 'sum',
    'osrm_distance': 'sum',
    'factor': 'mean',

    # segment_* columns
    'segment_actual_time': 'sum',
    'segment_osrm_time': 'sum',
    'segment_osrm_distance': 'sum',
    'segment_factor': 'mean'
}
```

In [272…

```
segment = df.groupby("segment_key").agg(create_segment_dict).reset_index(
```

In [273…

```
segment
```

Out[273…

	segment_key	trip_creation_time	
0	trip-153671041653548748_IND209304AAA_IND000000ACB	2018-09-12 00:00:16.535741	thai
1	trip-153671041653548748_IND462022AAA_IND209304AAA	2018-09-12 00:00:16.535741	thai
2	trip-153671042288605164_IND561203AAB_IND562101AAA	2018-09-12 00:00:22.886430	tha
3	trip-153671042288605164_IND572101AAA_IND561203AAB	2018-09-12 00:00:22.886430	tha
4	trip-153671043369099517_IND000000ACB_IND160002AAC	2018-09-12 00:00:33.691250	thai
...	...	...	
26363	trip-153861115439069069_IND628204AAA_IND627657AAA	2018-10-03 23:59:14.390954	tha
26364	trip-153861115439069069_IND628613AAA_IND627005AAA	2018-10-03 23:59:14.390954	tha
26365	trip-153861115439069069_IND628801AAA_IND628204AAA	2018-10-03 23:59:14.390954	tha
26366	trip-153861118270144424_IND583119AAA_IND583101AAA	2018-10-03 23:59:42.701692	th
26367	trip-153861118270144424_IND583201AAA_IND583119AAA	2018-10-03 23:59:42.701692	th

26368 rows × 24 columns

In [274…

```
segment = segment.sort_values(by=['segment_key', 'od_end_time'])
segment
```

Out[274…

	segment_key	trip_creation_time	
0	trip-153671041653548748_IND209304AAA_IND000000ACB	2018-09-12 00:00:16.535741	thai
1	trip-153671041653548748_IND462022AAA_IND209304AAA	2018-09-12 00:00:16.535741	thai
2	trip-153671042288605164_IND561203AAB_IND562101AAA	2018-09-12 00:00:22.886430	tha
3	trip-153671042288605164_IND572101AAA_IND561203AAB	2018-09-12 00:00:22.886430	tha
4	trip-153671043369099517_IND000000ACB_IND160002AAC	2018-09-12 00:00:33.691250	thai
...	...	...	
26363	trip-153861115439069069_IND628204AAA_IND627657AAA	2018-10-03 23:59:14.390954	tha
26364	trip-153861115439069069_IND628613AAA_IND627005AAA	2018-10-03 23:59:14.390954	tha
26365	trip-153861115439069069_IND628801AAA_IND628204AAA	2018-10-03 23:59:14.390954	tha
26366	trip-153861118270144424_IND583119AAA_IND583101AAA	2018-10-03 23:59:42.701692	th
26367	trip-153861118270144424_IND583201AAA_IND583119AAA	2018-10-03 23:59:42.701692	th

26368 rows × 24 columns

In [275…
```
segment['od_time_diff_hour']=(segment['od_end_time']-segment['od_start_ti
```

In [276…
```
segment.drop(columns=['od_start_time','od_end_time'],inplace=True)
```

In [277…
```
segment['destination_name']
```

Out[277…

	destination_name
0	Gurgaon_Bilaspur_HB (Haryana)
1	Kanpur_Central_H_6 (Uttar Pradesh)
2	Chikblapur_ShntiSgr_D (Karnataka)
3	Doddablpur_ChikaDPP_D (Karnataka)
4	Chandigarh_Mehmdpur_H (Punjab)
...	...
26363	Thisayanvilai_UdnkdiRD_D (Tamil Nadu)
26364	Tirunelveli_VdkkuSrt_I (Tamil Nadu)
26365	Tirchchndr_Shnmgprm_D (Tamil Nadu)
26366	Bellary_Dc (Karnataka)
26367	Sandur_WrdN1DPP_D (Karnataka)

26368 rows × 1 columns

dtype: object

In [278…

```
segment['destination_name'].str.split('_').str.len().value_counts()
```

Out[278…

destination_name	count
3	20882
4	2362
2	2055
1	1069

dtype: int64

splitting by source and destination

In [279…

```
segment['dstate'] = segment['destination_name'].str.extract(r'\((.*?)\)')
segment['city_place_code'] = segment['destination_name'].str.replace(r'\s

split_df = segment['city_place_code'].str.split('_', expand=True)
split_df = split_df.rename(columns={0:'dcity',1:'dplace',2:'dcode'})

segment = pd.concat([segment, split_df], axis=1)
```

In [280…

```
segment['sstate'] = segment['source_name'].str.extract(r'\((.*?)\)')
segment['city_place_code'] = segment['source_name'].str.replace(r'\s*\(.*

split_df = segment['city_place_code'].str.split('_', expand=True)
```

```
split_df = split_df.rename(columns={0:'scity',1:'splace',2:'scode'})

segment = pd.concat([segment, split_df], axis=1)
```

In [281…

```
segment
```

Out[281…

	segment_key	trip_creation_time	
0	trip-153671041653548748_IND209304AAA_IND000000ACB	2018-09-12 00:00:16.535741	thai
1	trip-153671041653548748_IND462022AAA_IND209304AAA	2018-09-12 00:00:16.535741	thai
2	trip-153671042288605164_IND561203AAB_IND562101AAA	2018-09-12 00:00:22.886430	tha
3	trip-153671042288605164_IND572101AAA_IND561203AAB	2018-09-12 00:00:22.886430	tha
4	trip-153671043369099517_IND000000ACB_IND160002AAC	2018-09-12 00:00:33.691250	thai
...	...	...	
26363	trip-153861115439069069_IND628204AAA_IND627657AAA	2018-10-03 23:59:14.390954	tha
26364	trip-153861115439069069_IND628613AAA_IND627005AAA	2018-10-03 23:59:14.390954	tha
26365	trip-153861115439069069_IND628801AAA_IND628204AAA	2018-10-03 23:59:14.390954	tha
26366	trip-153861118270144424_IND583119AAA_IND583101AAA	2018-10-03 23:59:42.701692	th
26367	trip-153861118270144424_IND583201AAA_IND583119AAA	2018-10-03 23:59:42.701692	th

26368 rows × 34 columns

In [282…

```
segment.columns
```

Out[282…

```
Index([                   'segment_key',             'trip_creation_tim
e',
                   'route_schedule_uuid',                     'route_typ
e',
                            'trip_uuid',                 'source_cente
r',
                         'source_name',             'destination_cente
r',
                    'destination_name',         'start_scan_to_end_sca
n',
                           'is_cutoff',                 'cutoff_facto
r',
                    'cutoff_timestamp', 'actual_distance_to_destinatio
n',
                         'actual_time',                      'osrm_tim
e',
                       'osrm_distance',                        'facto
r',
                   'segment_actual_time',              'segment_osrm_tim
e',
                 'segment_osrm_distance',                 'segment_facto
r',
                     'od_time_diff_hour',                        'dstat
e',
                      'city_place_code',                         'dcit
y',
                              'dplace',                         'dcod
e',
                                     3,                        'sstat
e',
                              'scity',                        'splac
e',
                              'scode',
3],
      dtype='object')
```

In [283…

```
segment['trip_creation_time']
```

Out [283…

	trip_creation_time
0	2018-09-12 00:00:16.535741
1	2018-09-12 00:00:16.535741
2	2018-09-12 00:00:22.886430
3	2018-09-12 00:00:22.886430
4	2018-09-12 00:00:33.691250
...	...
26363	2018-10-03 23:59:14.390954
26364	2018-10-03 23:59:14.390954
26365	2018-10-03 23:59:14.390954
26366	2018-10-03 23:59:42.701692
26367	2018-10-03 23:59:42.701692

26368 rows × 1 columns

dtype: datetime64[ns]

Extract features like month, year, day,

In [284…

```
segment['year'] = segment['trip_creation_time'].dt.year
segment['month'] = segment['trip_creation_time'].dt.month
segment['day'] = segment['trip_creation_time'].dt.day
segment['trip_hour'] = segment['trip_creation_time'].dt.hour
```

In [285…

```
# create_trip_dict = {
#     'segment_actual_time': 'sum',
#     'segment_osrm_time': 'sum',
#     'segment_osrm_distance': 'sum',
#     'od_start_time': 'first',
#     'od_end_time': 'last',
#     'source_center': 'first',
#     'destination_center': 'last'
# }
```

In [286…

```
# Dictionary for trip-level aggregation
create_trip_dict = {
    'trip_creation_time': 'first',
    'route_schedule_uuid': 'first',
    'route_type': 'first',
    'sstate': 'first',
    'dstate': 'last',
    'scity': 'first',
    'dcity': 'last',
    'scode': 'first',
    'dcode': 'last',
    'od_time_diff_hour': 'sum',   # total time across segments
```

```
    # core metrics
    'actual_time': 'sum',
    'osrm_time': 'sum',
    'osrm_distance': 'sum',
    'segment_actual_time': 'sum',
    'segment_osrm_time': 'sum',
    'segment_osrm_distance': 'sum',
    'actual_distance_to_destination': 'max',
}

trip = (
    segment
    .groupby('trip_uuid')
    .agg(create_trip_dict)
    .reset_index()
)
```

In [287…

```
trip = segment.groupby('trip_uuid').agg(create_trip_dict)
trip
```

Out[287…

trip_uuid	trip_creation_time	route_schedule_uuid	route_type	
trip-153671041653548748	2018-09-12 00:00:16.535741	thanos::sroute:d7c989ba-a29b-4a0b-b2f4-288cdc6...	FTL	
trip-153671042288605164	2018-09-12 00:00:22.886430	thanos::sroute:3a1b0ab2-bb0b-4c53-8c59-eb2a2c0...	Carting	
trip-153671043369099517	2018-09-12 00:00:33.691250	thanos::sroute:de5e208e-7641-45e6-8100-4d9fb1e...	FTL	
trip-153671046011330457	2018-09-12 00:01:00.113710	thanos::sroute:f0176492-a679-4597-8332-bbd1c7f...	Carting	Ma
trip-153671052974046625	2018-09-12 00:02:09.740725	thanos::sroute:d9f07b12-65e0-4f3b-bec8-df06134...	FTL	
...	...	...	...	
trip-153861095625827784	2018-10-03 23:55:56.258533	thanos::sroute:8a120994-f577-4491-9e4b-b7e4a14...	Carting	
trip-153861104386292051	2018-10-03 23:57:23.863155	thanos::sroute:b30e1ec3-3bfa-4bd2-a7fb-3b75769...	Carting	
trip-153861106442901555	2018-10-03 23:57:44.429324	thanos::sroute:5609c268-e436-4e0a-8180-3db4a74...	Carting	
trip-153861115439069069	2018-10-03 23:59:14.390954	thanos::sroute:c5f2ba2c-8486-4940-8af6-d1d2a6a...	Carting	T
trip-153861118270144424	2018-10-03 23:59:42.701692	thanos::sroute:412fea14-6d1f-4222-8a5f-a517042...	FTL	

14817 rows × 17 columns

In [288…

```
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
```

numerical columns

In [289…

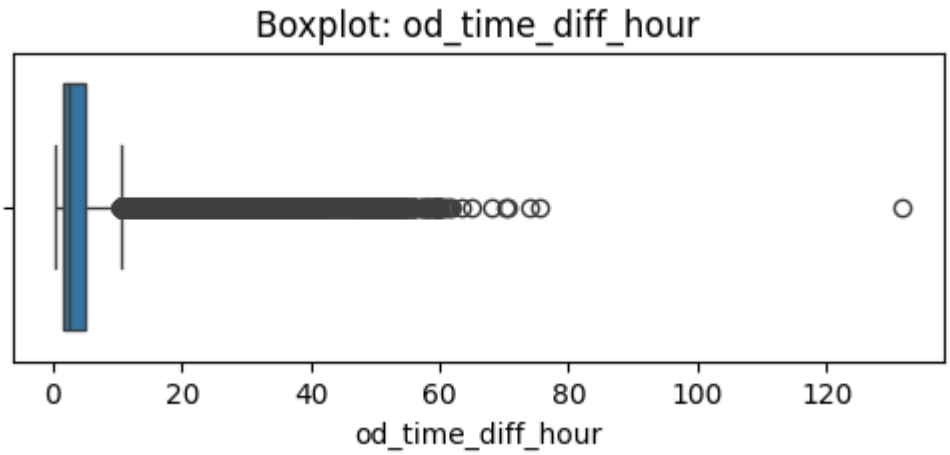
```
import matplotlib.pyplot as plt
import seaborn as sns

numeric_cols = [
    'od_time_diff_hour',
    'actual_time',
```

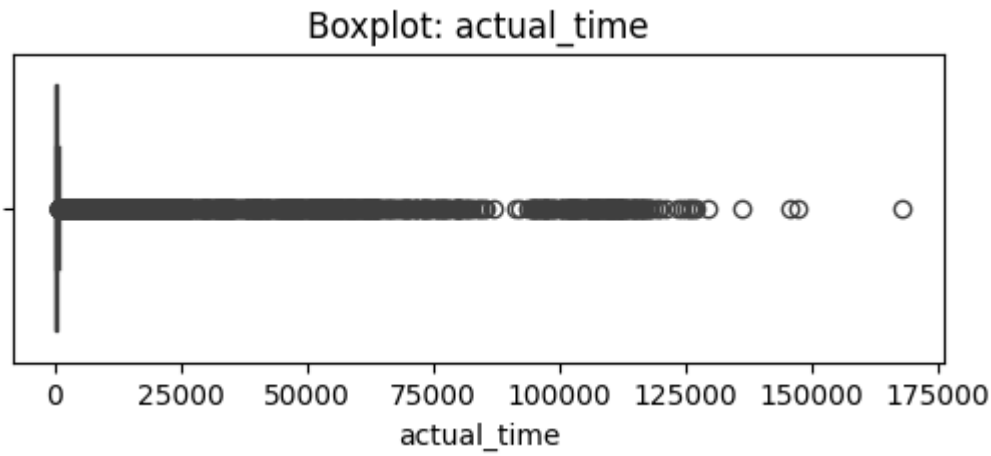
```
    'osrm_time',
    'osrm_distance',
    'segment_actual_time',
    'segment_osrm_time',
    'segment_osrm_distance',
    'actual_distance_to_destination'
]
```

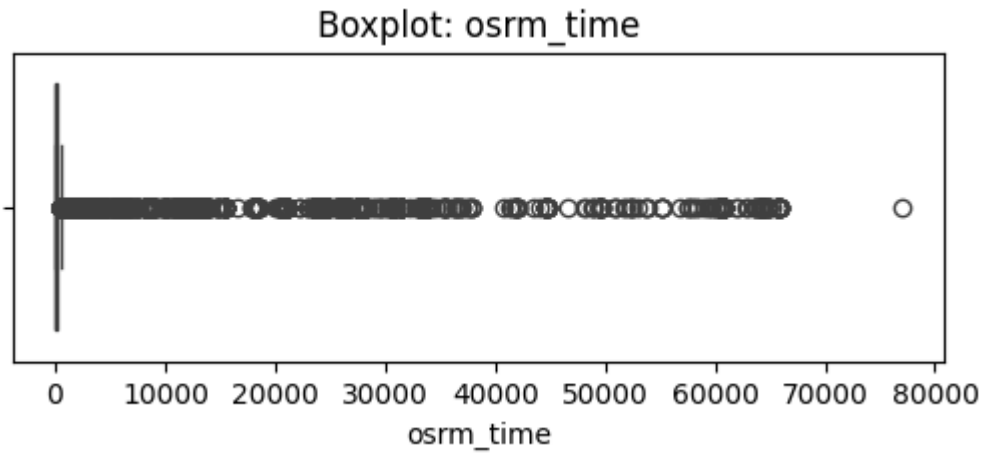
outliers

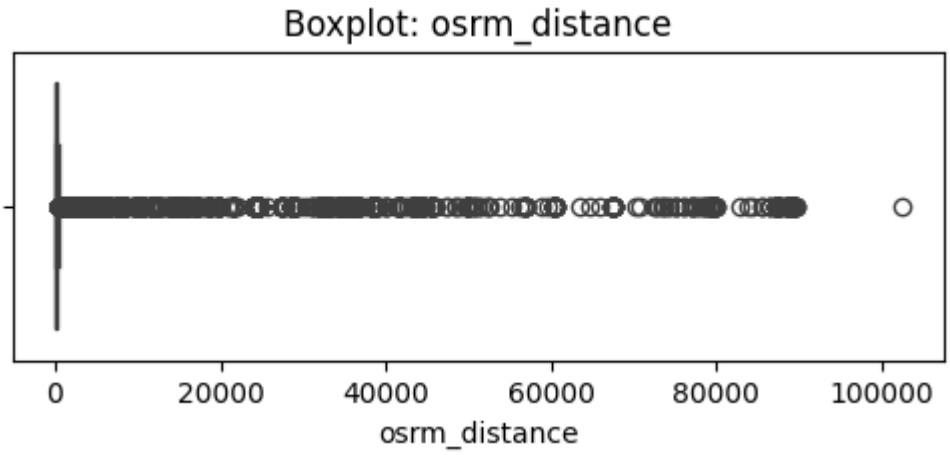
In [290…

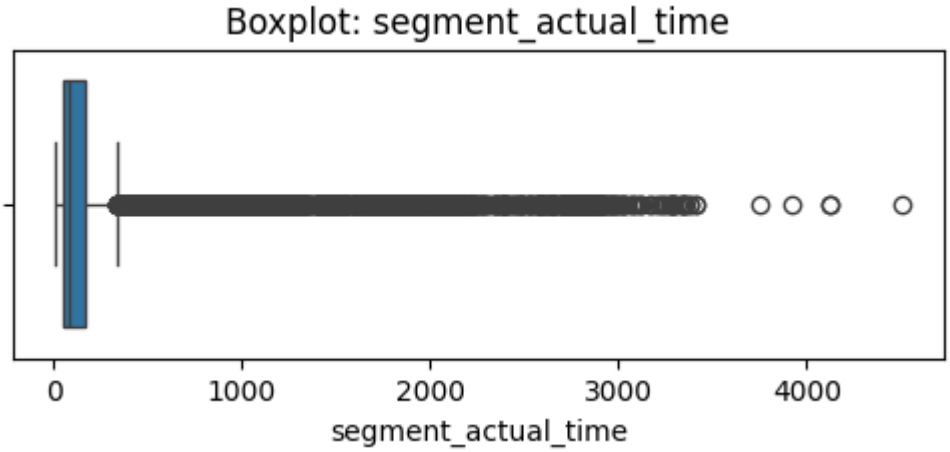
```
for col in numeric_cols:
    plt.figure(figsize=(6,2))
    sns.boxplot(x=segment[col])
    plt.title(f"Boxplot: {col}")
    plt.show()

# plt.figure(figsize=(14, 8))
# segment[numeric_cols].boxplot()
# plt.xticks(rotation=45)
# plt.show()
```

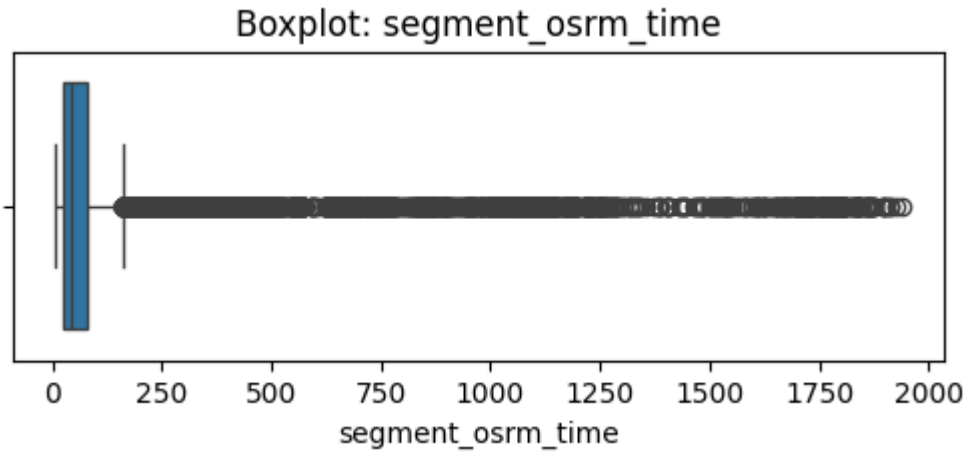


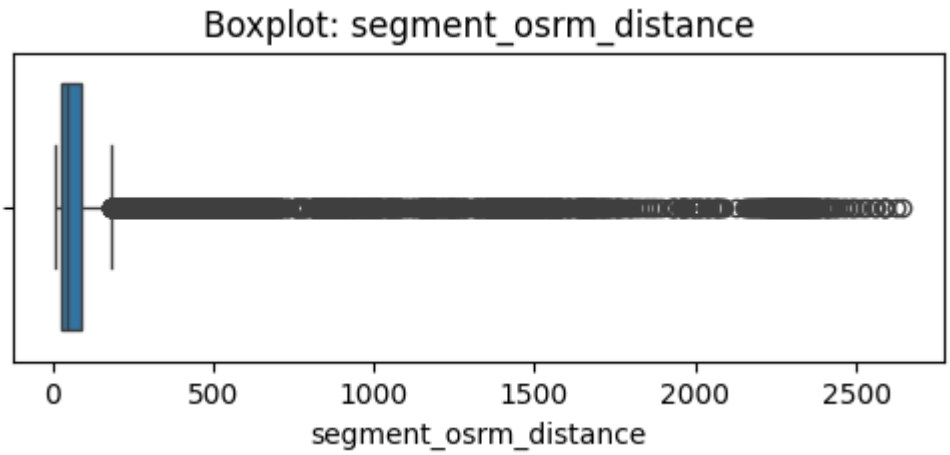


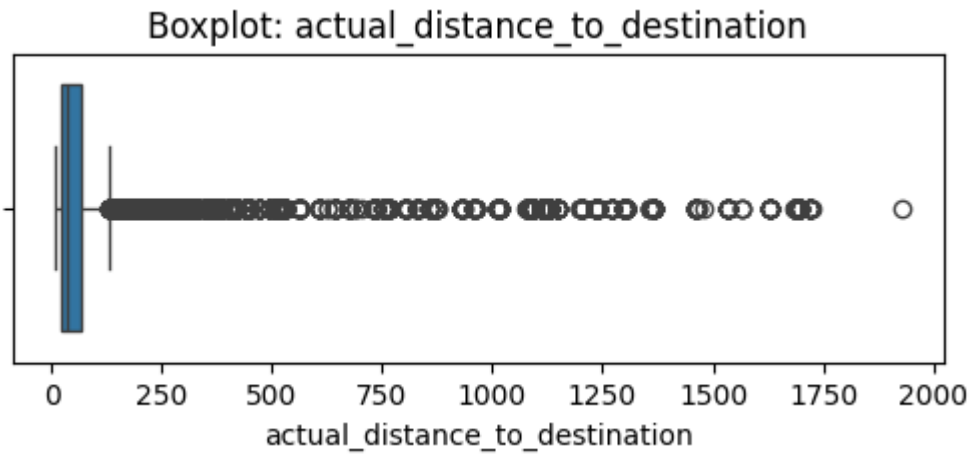

Boxplot: osrm_time
0
10000
20000
30000
40000
50000
60000
70000
80000
osrm_time


Boxplot: osrm_distance
0
20000
40000
60000
80000
100000
osrm_distance


Boxplot: segment_actual_time
0
1000
2000
3000
4000
segment_actual_time


Boxplot: segment_osrm_time
0
250
500
750
1000
1250
1500
1750
2000
segment_osrm_time





In [294…

```
for col in numeric_cols:
    Q1 = segment[col].quantile(0.25)
    Q3 = segment[col].quantile(0.75)
    IQR = Q3 - Q1

    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR

    print(f"{col}: lower={lower}, upper={upper}")
    filtered =segment.copy()
    # Remove or cap
    filtered[col] = segment[col].clip(lower, upper)
```

```
od_time_diff_hour: lower=-3.884356422638888, upper=10.519923291805554
actual_time: lower=-447.5, upper=988.5
osrm_time: lower=-211.0, upper=477.0
osrm_distance: lower=-235.991625, upper=543.812175
segment_actual_time: lower=-124.0, upper=340.0
segment_osrm_time: lower=-56.0, upper=160.0
segment_osrm_distance: lower=-65.8497125, upper=185.67298749999998
actual_distance_to_destination: lower=-44.747255308846825, upper=132.51800
740111221
```

In [295…

```
filtered
```

Out[295…

	segment_key	trip_creation_time	
0	trip-153671041653548748_IND209304AAA_IND000000ACB	2018-09-12 00:00:16.535741	thai
1	trip-153671041653548748_IND462022AAA_IND209304AAA	2018-09-12 00:00:16.535741	thai
2	trip-153671042288605164_IND561203AAB_IND562101AAA	2018-09-12 00:00:22.886430	tha
3	trip-153671042288605164_IND572101AAA_IND561203AAB	2018-09-12 00:00:22.886430	tha
4	trip-153671043369099517_IND000000ACB_IND160002AAC	2018-09-12 00:00:33.691250	thai
...	...	...	
26363	trip-153861115439069069_IND628204AAA_IND627657AAA	2018-10-03 23:59:14.390954	tha
26364	trip-153861115439069069_IND628613AAA_IND627005AAA	2018-10-03 23:59:14.390954	tha
26365	trip-153861115439069069_IND628801AAA_IND628204AAA	2018-10-03 23:59:14.390954	tha
26366	trip-153861118270144424_IND583119AAA_IND583101AAA	2018-10-03 23:59:42.701692	th
26367	trip-153861118270144424_IND583201AAA_IND583119AAA	2018-10-03 23:59:42.701692	th

26368 rows × 38 columns

In [296…

```
filtered.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 26368 entries, 0 to 26367
Data columns (total 38 columns):
 #   Column                          Non-Null Count  Dtype
---  ------                          --------------  -----
 0   segment_key                     26368 non-null  object
 1   trip_creation_time              26368 non-null  datetime64[ns]
 2   route_schedule_uuid             26368 non-null  object
 3   route_type                      26368 non-null  object
 4   trip_uuid                       26368 non-null  object
 5   source_center                   26368 non-null  object
 6   source_name                     26368 non-null  object
 7   destination_center              26368 non-null  object
 8   destination_name                26368 non-null  object
 9   start_scan_to_end_scan          26368 non-null  float64
 10  is_cutoff                       26368 non-null  bool
 11  cutoff_factor                   26368 non-null  int64
 12  cutoff_timestamp                26368 non-null  datetime64[ns]
 13  actual_distance_to_destination  26368 non-null  float64
 14  actual_time                     26368 non-null  float64
 15  osrm_time                       26368 non-null  float64
 16  osrm_distance                   26368 non-null  float64
 17  factor                          26368 non-null  float64
 18  segment_actual_time             26368 non-null  float64
 19  segment_osrm_time               26368 non-null  float64
 20  segment_osrm_distance           26368 non-null  float64
 21  segment_factor                  26368 non-null  float64
 22  od_time_diff_hour               26368 non-null  float64
 23  dstate                          26287 non-null  object
 24  city_place_code                 26368 non-null  object
 25  dcity                           26368 non-null  object
 26  dplace                          25299 non-null  object
 27  dcode                           23244 non-null  object
 28  3                               2362 non-null   object
 29  sstate                          26302 non-null  object
 30  scity                           26368 non-null  object
 31  splace                          25476 non-null  object
 32  scode                           23322 non-null  object
 33  3                               2508 non-null   object
 34  year                            26368 non-null  int32
 35  month                           26368 non-null  int32
 36  day                             26368 non-null  int32
 37  trip_hour                       26368 non-null  int32
dtypes: bool(1), datetime64[ns](2), float64(11), int32(4), int64(1), objec
t(19)
memory usage: 7.1+ MB
```

In [297…

```python
categorical_cols = [
    'route_type',
    'sstate',
    'dstate',
    'scity',
    'dcity',
    'scode',
    'dcode'
]

for col in categorical_cols:
    print(f"\nUnique values for {col}:")
    print(filtered[col].value_counts())
```

```
Unique values for route_type:
route_type
FTL        13939
Carting    12429
Name: count, dtype: int64

Unique values for sstate:
sstate
Maharashtra               3565
Karnataka                 3453
Tamil Nadu                2130
Haryana                   2056
Uttar Pradesh             1832
Telangana                 1484
Gujarat                   1401
West Bengal               1368
Andhra Pradesh            1310
Rajasthan                 1176
Bihar                     1060
Punjab                    1052
Delhi                      821
Kerala                     762
Madhya Pradesh             742
Assam                      484
Uttarakhand                369
Jharkhand                  301
Orissa                     248
Himachal Pradesh           223
Chandigarh                 160
Goa                         86
Chhattisgarh                52
Arunachal Pradesh           48
Jammu & Kashmir             47
Pondicherry                 30
Dadra and Nagar Haveli      15
Meghalaya                   13
Mizoram                      8
Nagaland                     5
Tripura                      1
Name: count, dtype: int64

Unique values for dstate:
dstate
Karnataka                 3505
Maharashtra               3473
Tamil Nadu                2111
Haryana                   2019
Uttar Pradesh             1848
Telangana                 1542
Gujarat                   1401
West Bengal               1399
Andhra Pradesh            1315
Rajasthan                 1210
Punjab                    1132
Bihar                     1055
Madhya Pradesh             750
Kerala                     746
Delhi                      703
Assam                      465
Uttarakhand                367
```

```
Jharkhand                  309
Orissa                     260
Himachal Pradesh           241
Chandigarh                 123
Goa                         86
Arunachal Pradesh           55
Chhattisgarh                52
Jammu & Kashmir             45
Pondicherry                 31
Dadra and Nagar Haveli      17
Meghalaya                   13
Mizoram                     10
Tripura                      2
Nagaland                     1
Daman & Diu                  1
Name: count, dtype: int64

Unique values for scity:
scity
Gurgaon         1141
Bengaluru       1136
Bhiwandi         821
Bangalore        792
Mumbai           719
                ...
Kothanalloor       1
Shahjhnpur         1
Basta              1
Anuppur            1
Kapadvanj          1
Name: count, Length: 1250, dtype: int64

Unique values for dcity:
dcity
Bengaluru    1180
Mumbai       1000
Gurgaon       986
Bangalore     683
Hyderabad     643
             ...
Falna           1
AmaDubi         1
Ranaghat        1
Kapadvanj       1
Lunawada        1
Name: count, Length: 1246, dtype: int64

Unique values for scode:
scode
D        9847
H        3674
HB       3531
I        2433
IP        732
DC        721
DPC       495
L         458
DPP       373
PC        313
C         255
```

```
CP        124
Dc         82
V          58
M          58
P          58
RP         39
Pc         36
Nagar      21
R          14
Name: count, dtype: int64

Unique values for dcode:
dcode
D          9795
H          3586
HB         2737
I          2609
IP          948
DC          827
DPC         797
L           639
DPP         353
P           245
PC          243
Dc          161
M            72
Pc           69
Nagar        49
R            36
GW           31
C            23
RPC          11
Layout        9
Gateway       2
INT           1
CP            1
Name: count, dtype: int64
```

Encoding

In [298…

```
categorical_cols = [
    'route_type',
    'sstate',
    'dstate',
    'scity',
    'dcity',
    'scode',
    'dcode'
]

filtered_encoded = pd.get_dummies(filtered, columns=categorical_cols, dro
```

In [299…

```
display(filtered_encoded.head())
```

	segment_key	trip_creation_time	route_s
0	trip-153671041653548748_IND209304AAA_IND000000ACB	2018-09-12 00:00:16.535741	thanos::sro a2
1	trip-153671041653548748_IND462022AAA_IND209304AAA	2018-09-12 00:00:16.535741	thanos::sro a2
2	trip-153671042288605164_IND561203AAB_IND562101AAA	2018-09-12 00:00:22.886430	thanos::sro bb0
3	trip-153671042288605164_IND572101AAA_IND561203AAB	2018-09-12 00:00:22.886430	thanos::sro bb0
4	trip-153671043369099517_IND000000ACB_IND160002AAC	2018-09-12 00:00:33.691250	thanos::sro 76

5 rows × 2628 columns

In [300…
```
filtered_encoded.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 26368 entries, 0 to 26367
Columns: 2628 entries, segment_key to dcode_RPC
dtypes: bool(2598), datetime64[ns](2), float64(11), int32(4), int64(1), ob
ject(12)
memory usage: 71.0+ MB
```

Scaling

In [302…
```
from sklearn.preprocessing import StandardScaler


# Initialize the StandardScaler
scaler = StandardScaler()

# Create a copy of the DataFrame to store scaled values
filtered_scaled = filtered_encoded.copy()

# Apply StandardScaler to the identified numerical columns
filtered_scaled[numeric_cols] = scaler.fit_transform(filtered_scaled[nume
```

In [303…
```
display(filtered_scaled.head())
```

	segment_key	trip_creation_time	route_s
0	trip-153671041653548748_IND209304AAA_IND000000ACB	2018-09-12 00:00:16.535741	thanos::sro a2
1	trip-153671041653548748_IND462022AAA_IND209304AAA	2018-09-12 00:00:16.535741	thanos::sro a2
2	trip-153671042288605164_IND561203AAB_IND562101AAA	2018-09-12 00:00:22.886430	thanos::sro bb0
3	trip-153671042288605164_IND572101AAA_IND561203AAB	2018-09-12 00:00:22.886430	thanos::sro bb0
4	trip-153671043369099517_IND000000ACB_IND160002AAC	2018-09-12 00:00:33.691250	thanos::sro 76

5 rows × 2628 columns

In [304…
```
filtered_scaled.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 26368 entries, 0 to 26367
Columns: 2628 entries, segment_key to dcode_RPC
dtypes: bool(2598), datetime64[ns](2), float64(11), int32(4), int64(1), ob
ject(12)
memory usage: 71.0+ MB
```

In [ ]:
```
trip1
```

In [ ]:
```
trip['source_center'].value_counts()
```

In [ ]:

Hypothesis testing

In [ ]:

Task

Perform paired t-tests for `actual_time` vs `osrm_time`, `actual_time` vs `segment_actual_time`, `osrm_distance` vs `segment_osrm_distance`, and `osrm_time` vs `segment_osrm_time` in the `trip` DataFrame. Then, create scatter plots to visualize these comparisons, and finally, summarize the findings from both the statistical tests and the visualizations.

Perform Hypothesis Tests

Subtask:

For each specified pair of aggregated metrics (actual_time vs OSRM time, actual_time vs segment actual time, OSRM distance vs segment OSRM distance, OSRM time vs segment OSRM time), perform a paired t-test to determine if there is a statistically significant difference between them. This will use the already aggregated data in the `trip` DataFrame.

Reasoning: To perform paired t-tests, I need to import the `ttest_rel` function from the `scipy.stats` module. Then, I will apply this function to the specified pairs of columns in the `trip` DataFrame and print the results.

In [305…

```
from scipy.stats import ttest_rel

# 1. actual_time vs osrm_time
t_stat_1, p_val_1 = ttest_rel(trip['actual_time'], trip['osrm_time'])
print(f"Paired t-test (actual_time vs osrm_time):\n  T-statistic = {t_sta

# 2. actual_time vs segment_actual_time
t_stat_2, p_val_2 = ttest_rel(trip['actual_time'], trip['segment_actual_t
print(f"Paired t-test (actual_time vs segment_actual_time):\n  T-statisti

# 3. osrm_distance vs segment_osrm_distance
t_stat_3, p_val_3 = ttest_rel(trip['osrm_distance'], trip['segment_osrm_d
print(f"Paired t-test (osrm_distance vs segment_osrm_distance):\n  T-stat

# 4. osrm_time vs segment_osrm_time
t_stat_4, p_val_4 = ttest_rel(trip['osrm_time'], trip['segment_osrm_time'
print(f"Paired t-test (osrm_time vs segment_osrm_time):\n  T-statistic =
```

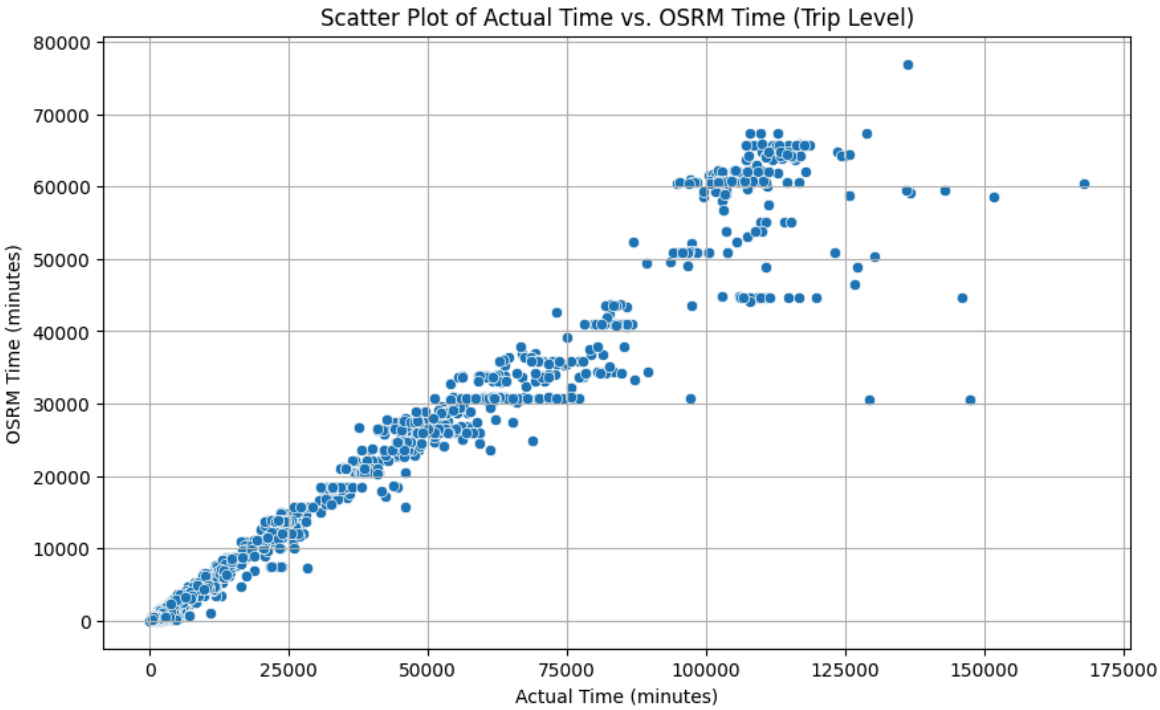
```
Paired t-test (actual_time vs osrm_time):
  T-statistic = 32.4681
  P-value = 1.8633e-223

Paired t-test (actual_time vs segment_actual_time):
  T-statistic = 30.7555
  P-value = 2.0773e-201

Paired t-test (osrm_distance vs segment_osrm_distance):
  T-statistic = 30.0303
  P-value = 2.1754e-192

Paired t-test (osrm_time vs segment_osrm_time):
  T-statistic = 30.2974
  P-value = 1.0893e-195
```

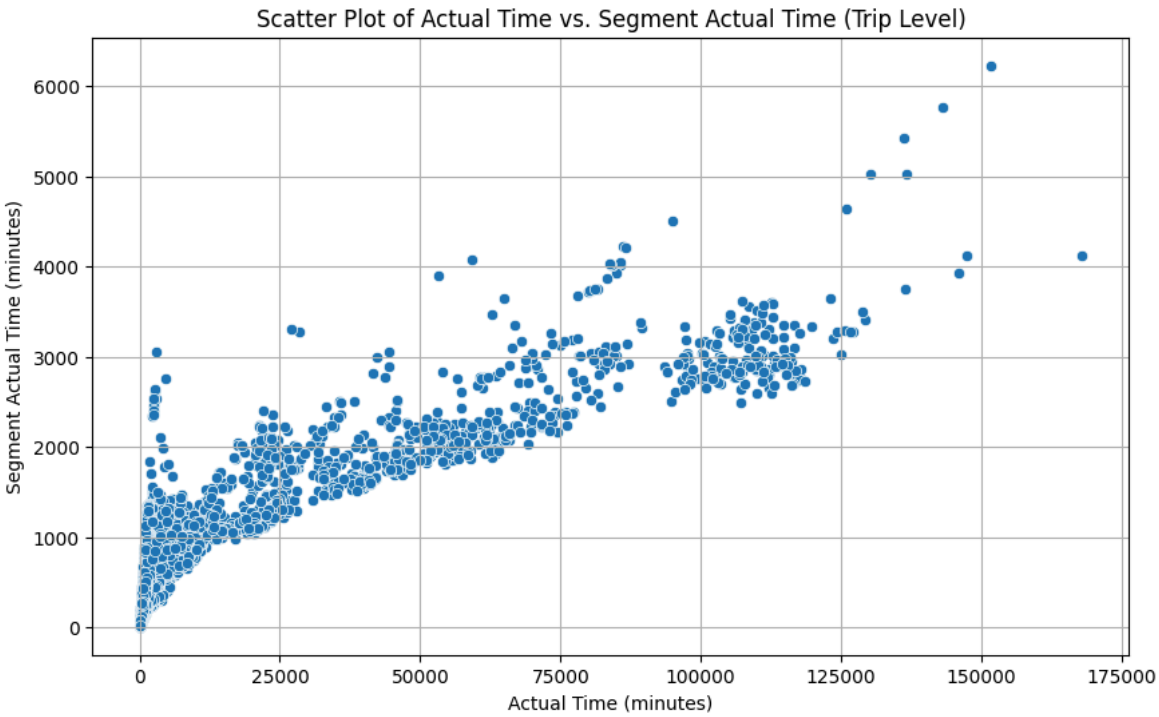
Reasoning: Now that the paired t-tests have been performed, the next step is to visualize these comparisons using scatter plots to provide a graphical representation of the relationships between the paired metrics. This will help confirm the statistical findings.

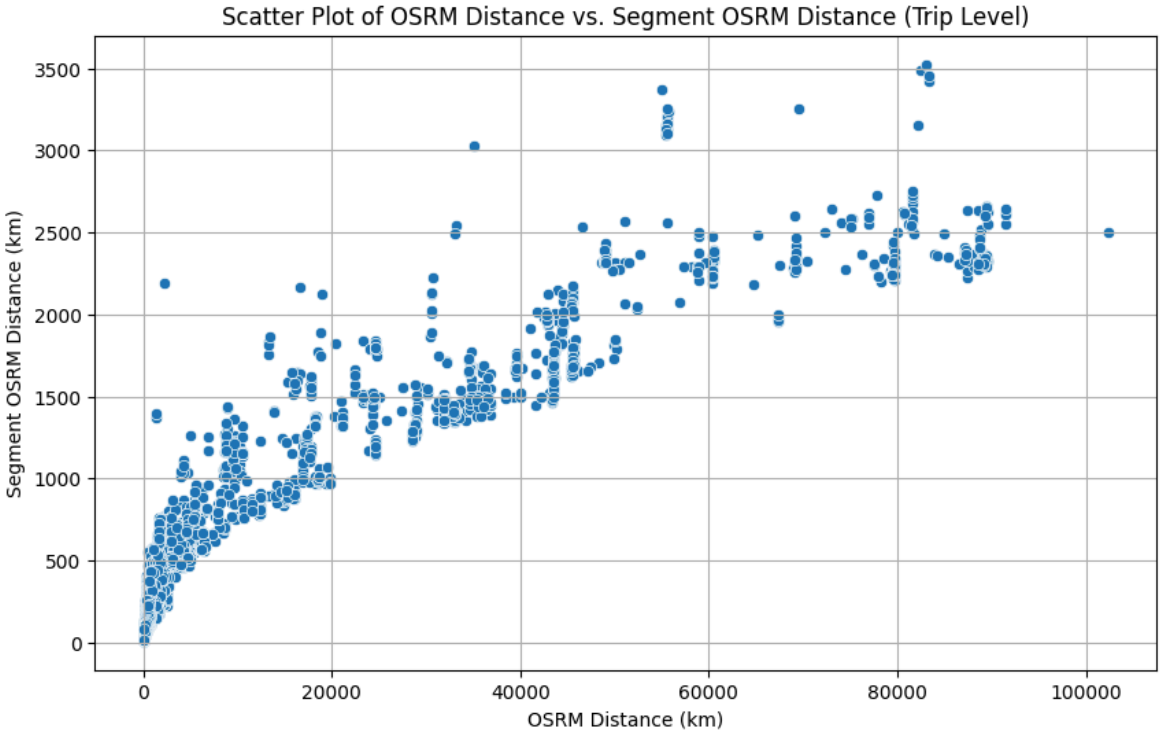
In [306…

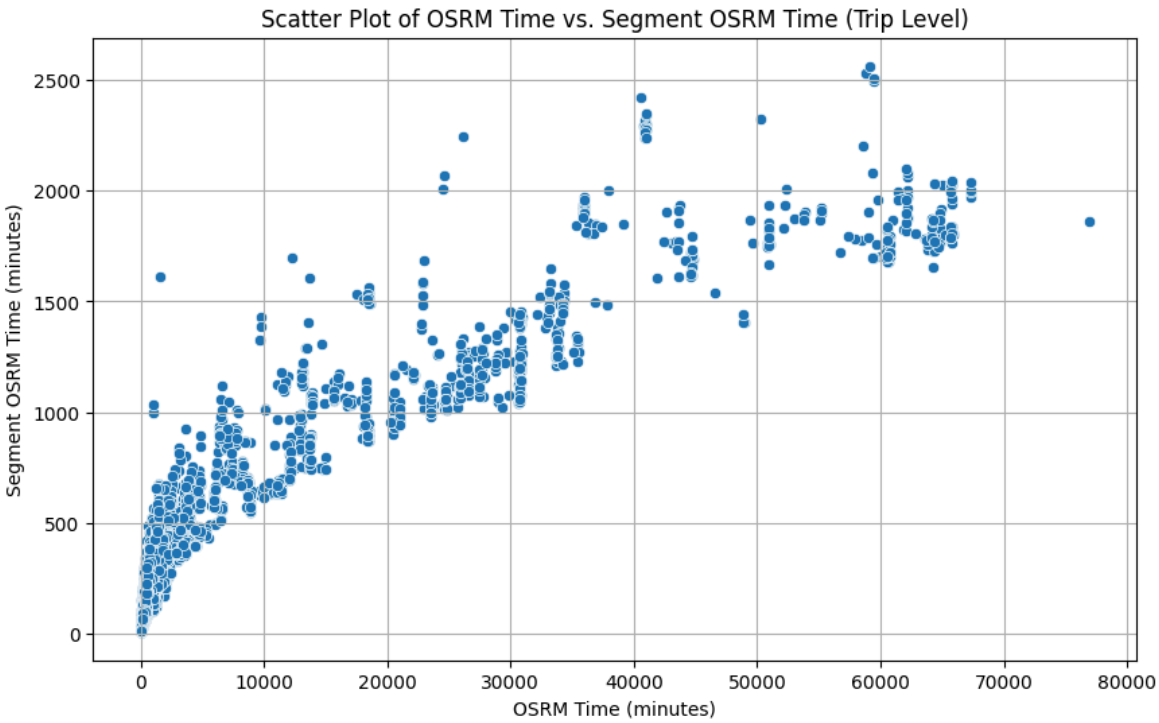
```
import matplotlib.pyplot as plt
import seaborn as sns

# 1. Scatter plot: actual_time vs osrm_time
plt.figure(figsize=(10, 6))
```

```
sns.scatterplot(x=trip['actual_time'], y=trip['osrm_time'])
plt.title('Scatter Plot of Actual Time vs. OSRM Time (Trip Level)')
plt.xlabel('Actual Time (minutes)')
plt.ylabel('OSRM Time (minutes)')
plt.grid(True)
plt.show()

# 2. Scatter plot: actual_time vs segment_actual_time
plt.figure(figsize=(10, 6))
sns.scatterplot(x=trip['actual_time'], y=trip['segment_actual_time'])
plt.title('Scatter Plot of Actual Time vs. Segment Actual Time (Trip Leve
plt.xlabel('Actual Time (minutes)')
plt.ylabel('Segment Actual Time (minutes)')
plt.grid(True)
plt.show()

# 3. Scatter plot: osrm_distance vs segment_osrm_distance
plt.figure(figsize=(10, 6))
sns.scatterplot(x=trip['osrm_distance'], y=trip['segment_osrm_distance'])
plt.title('Scatter Plot of OSRM Distance vs. Segment OSRM Distance (Trip
plt.xlabel('OSRM Distance (km)')
plt.ylabel('Segment OSRM Distance (km)')
plt.grid(True)
plt.show()

# 4. Scatter plot: osrm_time vs segment_osrm_time
plt.figure(figsize=(10, 6))
sns.scatterplot(x=trip['osrm_time'], y=trip['segment_osrm_time'])
plt.title('Scatter Plot of OSRM Time vs. Segment OSRM Time (Trip Level)')
plt.xlabel('OSRM Time (minutes)')
plt.ylabel('Segment OSRM Time (minutes)')
plt.grid(True)
plt.show()
```




Scatter Plot of Actual Time vs. Segment Actual Time (Trip Level)
Segment Actual Time (minutes)
Actual Time (minutes)
0
1000
2000
3000
4000
5000
6000
0
25000
50000
75000
100000
125000
150000
175000


Scatter Plot of OSRM Distance vs. Segment OSRM Distance (Trip Level)
Segment OSRM Distance (km)
OSRM Distance (km)
0
500
1000
1500
2000
2500
3000
3500
0
20000
40000
60000
80000
100000



Summary of Findings

Paired t-tests Analysis: The p-values for all four paired t-tests are extremely small (e.g., 1.8633e-223, 2.0773e-201, 2.1754e-192, 1.0893e-195). Since all p-values are significantly less than the common significance level of 0.05, we reject the null hypothesis for each test. This indicates that there is a statistically significant difference between the means of the paired metrics in all cases.

Scatter Plot Visualizations:

1. **Actual Time vs. OSRM Time:** The scatter plot shows a strong positive correlation between `actual_time` and `osrm_time`. However, most points lie above the line where `actual_time` equals `osrm_time`, suggesting that actual travel times tend to be higher than OSRM estimated times. This visual trend supports the statistical finding of a significant difference, indicating that OSRM generally underestimates the actual time.

2. **Actual Time vs. Segment Actual Time:** This scatter plot also exhibits a strong positive correlation. The values for `segment_actual_time` (which is the sum of individual segment actual times for a trip) are generally lower than `actual_time` (the total actual time for the trip, which might include non-travel times). The points show a spread, and a clear diagonal line is not observed, confirming a significant difference between these two metrics.

3. **OSRM Distance vs. Segment OSRM Distance:** A very strong positive linear relationship is visible here, with most points closely clustering around a diagonal line. This suggests that `osrm_distance` and `segment_osrm_distance` (sum of individual segment OSRM distances) are very similar, which is expected as both are OSRM-based distance calculations. While the t-test still found a significant difference, the visual difference is less pronounced compared to time

metrics, implying that the magnitude of the difference might be small or due to subtle aggregation methods.

4. **OSRM Time vs. Segment OSRM Time:** Similar to the distance plots, this scatter plot shows a strong positive correlation with points generally following a diagonal trend. `osrm_time` and `segment_osrm_time` (sum of individual segment OSRM times) are quite close. Again, while the t-test indicated a significant difference, the visual representation suggests that these two OSRM-based time metrics are largely consistent with each other.

Overall Conclusion: The hypothesis tests consistently show statistically significant differences across all compared pairs. The scatter plots provide visual confirmation of these differences and illustrate the nature of the discrepancies. Specifically, `actual_time` tends to be higher than OSRM estimations (`osrm_time` and `segment_actual_time`), suggesting that the model needs to account for factors beyond just OSRM estimates for accurate time prediction. The OSRM-based distance and time metrics (`osrm_distance` vs `segment_osrm_distance` and `osrm_time` vs `segment_osrm_time`) are highly correlated and visually similar, implying internal consistency within the OSRM calculations, even if small statistical differences exist.

Summary:

Data Analysis Key Findings

- **Statistical Significance Across All Comparisons:** Paired t-tests revealed a statistically significant difference
- **OSRM Underestimates Actual Travel
- **High Internal Consistency within OSRM Metrics:** Both `osrm_distance` vs `segment_osrm_distance` and `osrm_time` vs `segment_osrm_time` showed very strong positive correlations, with points closely aligned to the diagonal. This indicates high consistency between the overall OSRM calculations and the sum of their respective segment-level calculations, despite the statistical significance of small differences.

Data Where most orders are coming from

In [7]:
```
trip['sstate'].value_counts().head(10)
```

Out[7]:

sstate	count
Maharashtra	2714
Karnataka	2143
Haryana	1838
Tamil Nadu	1039
Telangana	785
Uttar Pradesh	762
Gujarat	750
Delhi	728
West Bengal	665
Punjab	536

dtype: int64

most orders come from maharastra followed by karnataka

In [9]:
```
trip['corridor'] = trip['sstate'] + " → " + trip['dstate']
trip['corridor'].value_counts().head(10)
```

Out[9]:

corridor	count
Maharashtra → Maharashtra	2453
Karnataka → Karnataka	2057
Tamil Nadu → Tamil Nadu	1021
Haryana → Haryana	949
Telangana → Telangana	689
Gujarat → Gujarat	629
West Bengal → West Bengal	615
Uttar Pradesh → Uttar Pradesh	597
Rajasthan → Rajasthan	453
Andhra Pradesh → Andhra Pradesh	414

dtype: int64

most orders in corridor are maharastra,karnataka,tamilnadu

In [11]:
```
corridor_stats = (
    trip.groupby('corridor')
        .agg(
```

```
            trips=('trip_uuid','count'),
            avg_distance=('osrm_distance','mean'),
            avg_time=('od_time_diff_hour','mean')
        )
        .sort_values('trips', ascending=False)
)
```

In [12]:
```
corridor_stats
```

Out[12]:

corridor	trips	avg_distance	avg_time
Maharashtra → Maharashtra	2453	511.911670	5.264808
Karnataka → Karnataka	2057	254.795466	4.720377
Tamil Nadu → Tamil Nadu	1021	238.423115	4.597784
Haryana → Haryana	949	7483.075281	11.612613
Telangana → Telangana	689	402.039681	7.315953
...	...	...	...
Tamil Nadu → Kerala	1	469.090100	5.414306
Telangana → Orissa	1	19567.900300	28.138868
Telangana → Haryana	1	48355.351700	45.746066
Uttar Pradesh → Uttarakhand	1	273.291800	4.298661
West Bengal → Andhra Pradesh	1	8831.379800	26.807990

141 rows × 3 columns

Business insights

*Optimize High-Volume Corridors

Actions:

Add dedicated fleets or line haul trucks for those routes.

Lock-in fixed capacity contracts or shared capacity.

Introduce priority batch loading windows to eliminate queue delays.

- Segment by Trip Type & Region

Patterns usually show:

Night shipments longer but more consistent

Peak daytime shipments shorter but more variable

Action:

Night logistics on long corridors → more reliable delivery time

Day runs for intra-city routes → faster turnaround and cheaper

- Rebalance Inventory Placement

If many shipments originate from a single hub, it is overloaded.

Action:

Create secondary consolidation hubs near high-demand destinations

Reduce the number of outbound “small trucks per many destinations”

Introduce hub–spoke routing based on region load

- Prioritize Predictive Planning

Features you engineered (month, day, hour) are gold:

Seasonality (festival spikes)

Weekday patterns

Hour-based congestion

Machine learning opportunity:

Predict delivery time buckets per corridor

Adaptive route allocation dynamically

- Operational Efficiency Recommendations

Batch shipments by destination clusters Schedule long hauls during low-traffic window

Allocate larger vehicles for high-volume corridors

Introduce cross-docking for busy corridors

Detect underperforming segments (high actual_time vs OSRM)

Summary Observations

Orders cluster heavily in certain states/corridors.

Top corridors carry majority volume — critical to overall performance.

Actual times > OSRM → indicates systemic real-world impact (loading, city traffic, hand-offs).

- Recommendations

Optimize high-volume corridors

Improve hub distribution

SLA based on historical actual performance

Time-window logistics routing

Predictive models for delay & resource allocation

- Summary Observations

Orders cluster heavily in certain states/corridors.

Top corridors carry majority volume — critical to overall performance.

Actual times > OSRM → indicates systemic real-world impact (loading, city traffic, hand-offs).

- Recommendations

Optimize high-volume corridors

Improve hub distribution

SLA based on historical actual performance

Time-window logistics routing

Predictive models for delay & resource allocation